Faster Than Light, According to the Arithmetic: Two Ways a Streaming Benchmark Fails on Sub-Millisecond Paths

Gustavo Pedro Ricou, Romaric Duvignau, Nikolas Herbst, and David Gregg

***Abstract*—On sub-millisecond paths a broker benchmark's instrument timescale can exceed the flight it reports, with two consequences invisible in the output. First, a measured latency can come out negative: both timestamps are written by threads that must first be scheduled, and the difference between those delays can outlast the delivery. A sign check rejected $1{,}321$ of our $2{,}266$ runs; the acknowledgment-referenced span inverts while the send-referenced span never does, on one clock by construction, so the cause is scheduling not synchronization, and real-time priority at fixed utilization collapses the rate 7–$80\times$. That span also stands for a delivery several times longer, distorting the two-broker gap a comparison exists for. Second, the benchmark the comparison literature shares admits an *end-to-end* sample only when a millisecond-quantized difference is positive and counts none of what it drops, while timing its own same-process span with a nanosecond clock. Of 75 embedded-mode cells, 71 reported a median on that millisecond grid — computed there from between 0.36% and 100% of the samples taken. We contribute a deletion law, a negative-span mechanism established by manipulation, and an audit of ten instrumentation tools. The remedy is one line: dither the send instant and publish that retained fraction.**

***Index Terms*—distributed systems; measurement techniques; measurement, evaluation, modeling, simulation of multiple-processor systems; message passing; performance attributes; scheduling and task partitioning.**

I. Introduction

STATED plainly, this paper reports two ways a latency benchmark can be wrong about its own measurements, and one number that governs both.

A latency benchmark times a message by reading a clock when it is sent and again when it arrives, then subtracting. Figure 1 is the system we built to do that and the four timestamps it takes. Nothing about the message takes those readings: two *threads* do (Fig. 2), each of which must first be given a core, and each of which waits for one on its own account. A *flight*, here and throughout, is that one span from send to arrival — not a clock period and not a sampling window — and on a busy machine either wait can outlast it. When the two waits *differ* by more than the flight, the subtraction goes negative: nothing arrived before it was sent, the two readings were simply taken in the wrong order. Separately, the benchmark the cross-broker comparison literature shares (Section II-A) timestamps its cross-process span in whole milliseconds and keeps only positive differences. On a path faster than a millisecond it therefore deletes samples at a rate fixed by arithmetic rather than chance. Neither failure appears in the output, and both are caught by checks that cost nothing.

The two failures share one number. Write T_{true} for the flight being measured and put beside it whichever timescale the instrument contributes: the scheduling stall before a timestamp is written, or the resolution of the timestamp itself. When that instrument timescale exceeds T_{true}, the measurement stops describing the system and starts describing the instrument. Sub-millisecond broker paths are where that ratio crosses one for tooling in current use.

That a measurement degrades as the flight shrinks is not the claim. The claim is about where the limit falls, and it does not fall where the datasheet says. The timestamp is one millisecond, but the timescale that binds is the scheduler's: the run-queue stall distribution's last mode sits at 3 ms (Section V-D), three times the quantum and nothing a specification discloses. A practitioner reasoning that a 0.5 ms path and a 1 ms timestamp are the same order of magnitude, so the measurement should be roughly sound, is wrong by a factor they had no way to look up. The instrument gives no sign when the line has been crossed.

A. Contributions

Three, each established by manipulation rather than by observation. **A deletion law** (Section IV): what the guard deletes is not noise but arithmetic, the retained fraction being fixed by the ratio of send interval to timestamp quantum and never shown to the operator. Across seven denominators, 10 of 12 commensurate arms behave as positioned. The two that do not are the superseded spread statistic's known failure mode, and its replacement rejects the continuum on both. A pre-registered payload manipulation moves an arm onto a grid vertex and off it again. **The negative-span mechanism, on one clock** (Section V): the acknowledgment-referenced span is the delivery less the acknowledgment lag, $S = D - A$ per event. A negative latency is therefore an ordering between two quantities we measure, not a fault in either clock (Equation 3). Four manipulations separate it from its rivals, and the send-referenced span over the same $738{,}730$ events never inverts,

G. P. Ricou and D. Gregg are with the School of Computer Science and Statistics, Trinity College Dublin, Dublin 2, Ireland. E-mail: pedrorig@tcd.ie; david.gregg@tcd.ie.

R. Duvignau is with the Department of Computer Science and Engineering, Chalmers University of Technology, Gothenburg, Sweden. E-mail: duvignau@chalmers.se.

N. Herbst is with the Chair of Computer Science II, University of Würzburg, Würzburg, Germany. E-mail: nikolas.herbst@uni-wuerzburg.de.

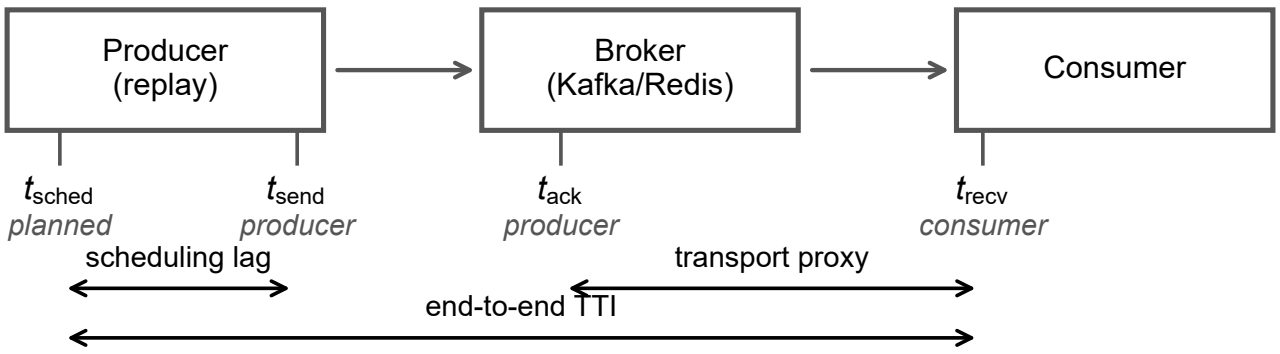

Fig. 1. **The system measured, and where its clocks are read.** A replay process paces recorded match events at a chosen rate into a broker — Kafka or Redis Streams — and a consumer reads them back. Four timestamps bracket the spans drawn here: t_{sched}, when an event was due; t_{send}, immediately before the send call; t_{ack}, when the producer learns the broker accepted it; and t_{recv}, when the consumer holds the record. A fifth, t_{out}, is consumer-internal (Section III-A). The transport proxy, the span this paper is about, subtracts a producer-process timestamp from a consumer-process one. Both processes run on one host reading one clock unless stated otherwise (Section III-A), so nothing that follows depends on clock synchronization.

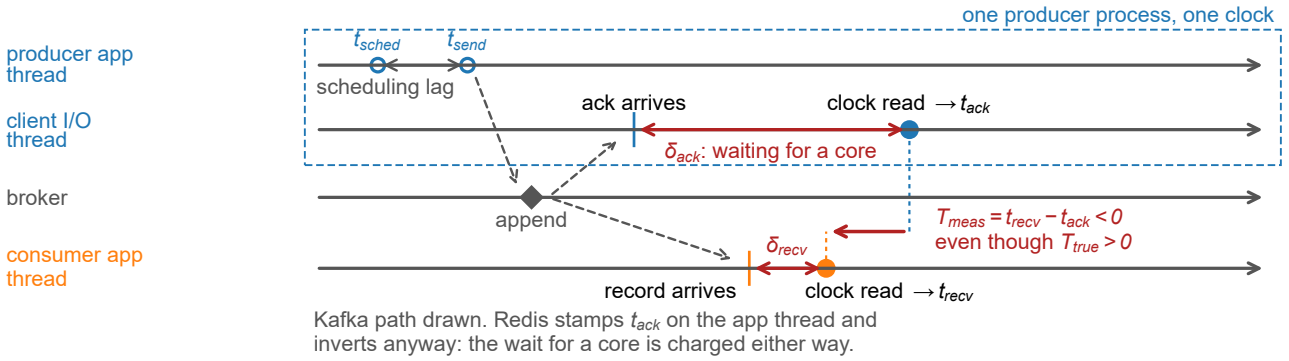

Fig. 2. **A late timestamp, not an early record.** How a positive latency is measured as negative, on one clock. Two threads of the producer process take the two timestamps — the application thread calls send, the client's I/O thread runs the delivery callback — and whichever one timestamps must first be scheduled, so the difference inverts when the acknowledgment's timestamping delay exceeds the true delivery plus the consumer's (Section III-A). The path drawn is Kafka's, and the figure's own note covers Redis, which has no second thread and inverts on a larger share of runs (Supplement S43.3). The producer's own timestamps, t_{sched} and t_{send}, do precede the acknowledgment, and the span referenced to them never inverts (Table I).

locating the fault in the reference timestamp rather than in the system under test. **An audit and the rules that follow** (Sections III-B and VI-B): a sign check applied uniformly to all 2,266 runs rejected 1,321 of them, including every run behind our own first result. Each part of that practice is established already [1], [2], [3]. What we add is their conjunction — a physical-impossibility criterion, applied per run, with the rejection rate published as a headline quantity — and the observation that the retention rate *of samples* and the sign of each discard belong beside every reported latency.

B. Chronology, and what was withdrawn

We set out to compare two brokers on a live sports feed. The audit of Section III-B rejected our own first answer — a complete, statistically significant result set — and a later audit refuted a claim we had already published about the benchmark. What follows is what survived. Supplement S35 has the order of discovery, the withdrawn claims and the failed pre-registrations, an audit hiding its own failures being worth less than one that does not.

Section VI-B is the rule list the two failures produce.

II. Background and Related Work

The claim that measurement can dominate a benchmark is not new; what is missing is the practice of checking for it run by run.

A. Streaming benchmarks

Broker comparisons appear continuously, in peer-reviewed venues and in the gray literature practitioners read. Apache Kafka and Redis Streams are the pair they most often name, and the OpenMessaging Benchmark [4] is the shared instrument; Supplement S52.3 sizes that literature. That experimental setup can dominate a result is established [5], and how to report one is a settled literature we follow [6], [7]. Most of those comparisons report a median above the quantum, where the law of Section IV does not bind. That is the scope of our claim rather than a limit on its reach — but not all of them are above it, and a 2026 comparison that calls two of three

brokers sub-millisecond without naming a percentile cannot be placed against the quantum at all (Supplement S52.3).

The instruments are few and shared, so a defect in one is not one project's problem. The audit of Section IV-D reads ten tools at source and finds five disposing of inverted samples without counting them. The quantum reaches further than the disposal does: the end-to-end tool bundled with Apache Kafka itself computes its percentiles from millisecond-truncated integers [8]. The architecture that produces those spans is not a blunder to be designed away — a producer that blocked for each acknowledgment could not generate load at all, and no design that keeps the asynchrony has one thread to timestamp with (Supplement S38). What a benchmark can drop is the habit of subtracting two such timestamps without ever checking the sign of the result.

That literature already stands its clocks carefully: Karimov et al. [9] separate event-time from processing-time latency, and Van Dongen and Van den Poel [10] use single-broker append timestamps "to ensure correct latency measurements" — our one-clock rule, arrived at independently. Neither asks what a millisecond timestamp does to a sub-millisecond path, nor checks the sample the instrument keeps; those two questions are this paper (Supplement S18).

B. Cross-process time measurement

Lamport [11] established that a distributed system has no single physical time, only a causal partial order. That order decides this paper's central span: an acknowledgment at the producer and a record at the consumer are two branches from the broker's append, and neither precedes the other. Instruments are built to beat clock disagreement [12] by calibrating cross-node counters [13], timestamping in the NIC [14] or the switch dataplane [15], and measuring the timer's own accuracy [16].

One prior result bounds our claim closely. Villain et al. [17], profiling FreeBSD against a DAG hardware reference, found the outgoing socket timestamp "does not respect causality" — packets leave before the timestamp meant to mark their departure is written — under every stress pattern including none. They attribute host latency to interrupt processing, scheduling and context switching. That is Mode A's physical primitive, published in 2012 and measured more precisely than we measure it. We add its consequence for a two-process measurement, a law relating the negative-span rate to the flight measured (Equation 6, Section V-B), and manipulations separating scheduling from its rivals. The audit half has its own ancestor. Paxson [18] treated physically impossible transit times as instrument failure and published the proportion of traces affected; Lancet is the nearest existing gate, reporting an inconclusive verdict where we decline to publish at all. Neither reports the fraction of runs a campaign lost.

Production tracing reached the same answer without a mechanism: skew adjusters that rewrite a span's timestamps are called harmful and default to off (Supplement S52.1). Where the displacement is a late timestamp rather than a skewed clock, correcting it moves evidence rather than error, and nobody had measured how much.

The nearest concurrent statement of the wider position is Swami and Chougule [19]: timing observability is itself an interference-sensitive resource. We agree, and need no apparatus for it. Their reference is a logic analyzer outside the system; ours is the sign of a subtraction inside it, available to any harness that already takes both timestamps.

Concurrent work reaches the same observable by another route. In a 2026 preprint, Sharma et al. [20] count negative timing spans in distributed inference systems and propose a `causality_health` check. Injecting skew, they see no violations up to 3 ms and clear ones by 5 ms, and read that as a skew threshold. We take their term, and dispute the reading rather than their measurements. They hold that "queueing alone cannot produce negative timing spans". For the queue they model, backlog at a service stage, we agree; but a *second* queue decides ours — the run queue the timestamping thread waits in for a CPU. We see rates near 23% at 88% utilization on one host, with an inter-host offset of 0.067 ms where two are used, and we move that rate fiftyfold without touching a clock. A threshold in milliseconds of skew does not transfer to a loaded machine (Supplement S50). The same attribution was offered for the benchmark we audit: asked why publish latency could exceed end-to-end latency, a maintainer cited inter-node skew of "~100ms" [21] and, told both processes ran on one node, called the measurement reliable. That is the arm our audit rejects most often.

C. Resolution and discarded samples

That a timer coarser than the flight invalidates it is textbook [16], and older than every tool we audit (Supplement S33). Metrology has long carried both our failure modes in one uncertainty budget: the NIST stopwatch guide budgets reaction time beside display resolution, and notes that at short flights the resolution becomes significant against the instrument's rated accuracy [22]. We claim neither the pairing nor its novelty. Nor do we claim the observation that periodic sampling commensurate with a periodic phenomenon sees only part of it: that is the IPPM framework [23], and RFC 3432 makes a randomized start mandatory for periodic streams [24]. Coordinated omission [25] is the mirror image — there the instrument fails to record samples that would have been slow, here it deletes samples that were fast. Tene's point, that omission correlated with the value measured skews the result, covers both. YCSB reported sub-millisecond operations as zero until it adopted microsecond histograms [26], and kept those samples rather than deleting them. Millsap and Holt draw the opposite conclusion from the same arithmetic [27]: coarse timing survives in aggregate *because* the two signed errors cancel. A guard admitting only positive samples removes one sign, and with it the cancellation that defence depends on (Supplement S41).

The mathematics is older still and is not ours. Counter metrology settled it for time-interval averaging half a century ago. Hewlett-Packard's Application Note 162-1 [28] carries our retention identity, our co-prime denominator, our ceiling on resolution gain, the symptom and the cure, all of it (Supplement S51). What is new is the sign of the operation: HP *keeps*

both readings and the retained fraction *is* the estimator, while the benchmark discards that population. Our contribution is not the law — we derived it, then found it in a counter manual — but its consequence under deletion. A harness released in 2018 throws away the quantity a 1970 counter measured with.

a) Sources of scheduling delay: A runnable thread does not always run, and work-conservation violations are documented for CFS-era schedulers [29], ours being EEVDF-era. Queueing gives the leading-order account, and the geometry contrast of Table II refutes a single-parameter law rather than queueing itself. A 2026 preprint by Chandrasekar and Kramberger [30] applies such a law to benchmark bias and reaches our inflation half independently; we refute it on the load axis (Section V). What a positive number cannot show remains ours: the negative span, and the sign channel that makes the displacement legible (Supplement S52.4).

III. Method

Everything below is measured on one clock unless the text says otherwise, and the spans we audit are named explicitly, because which two timestamps are subtracted is the whole question. A span whose difference comes out below zero we call a *negative span*. The term is Sharma et al.'s [20], taken deliberately: we keep it for the observable and decline the causal reading they give it, because what a negative span establishes is that one of two timestamps is untrustworthy, not which. *Span* is used in the tracing sense, a pair of timestamps bracketing a flight, and nothing here is specific to a tracing system; nor is this RFC 1242's negative latency, a cut-through forwarding artifact [31]. *Retention* is the fraction of the samples an instrument takes that survive into what it reports, not Kafka's log-retention setting.

A. Testbeds, and what the measurement proxies

Every finding comes from four Oracle Cloud `VM.Standard.E5.Flex` instances, three brokers and one driver. They run Ubuntu 22.04 on kernel `6.8.0-1057-oracle` (EEVDF scheduler) and CPython 3.10, on a private network, with client libraries pinned identically and `chrony`-disciplined clocks logged every minute. One early phase perturbed the measurement enough to be unusable and is excluded from every result here; the campaigns that remain measure utilization with no core pinning (Supplement S35). A second testbed, a workstation running the brokers under containers, produced our withdrawn first result and appears only as an audit outcome.

a) The measurement at a glance: The harness runs producer and consumer as separate *processes on one host*. Both timestamp the wall clock (`time.time_ns()`, CLOCK_REALTIME) rather than a monotonic counter or the TSC, since a cross-process span needs a shared epoch that no per-core cycle counter provides. The cost is worth naming. Only CLOCK_MONOTONIC guarantees consecutive reads do not go backwards, so we forgo an exclusion the clock could have supplied and establish it from data instead (Section V; Supplement S42 records what the probe did not log). The workload is a public football event feed (StatsBomb open data, CC BY-NC-SA 4.0) replayed against Kafka and Redis Streams. The send schedule is measured rather than assumed: pacer jitter is 67–69 μs at p90. The achieved rate is recorded per run against elapsed wall time — the self-validation SPEC and TPC ask of a benchmark [3], applied to the load generator rather than the system under test. The measured inter-host offset, where two hosts are used, is 0.067 ms. The primary measure is **Time-to-Insight** (TTI), from scheduled emission to availability at the consumer:

$$\mathrm{TTI} = \underbrace{(t_{\mathrm{send}} - t_{\mathrm{sched}})}_{\text{scheduling lag}} + \underbrace{(t_{\mathrm{ack}} - t_{\mathrm{send}})}_{\text{acknowledgment lag } A} + \underbrace{(t_{\mathrm{recv}} - t_{\mathrm{ack}})}_{\text{transport proxy } S}. \tag{1}$$

The three intervals telescope to $t_{\mathrm{recv}} - t_{\mathrm{sched}}$. A fifth timestamp sits outside them and has to be named, because the tables below subtract from it: the consumer writes t_{out} once it has the record in hand, and $t_{\mathrm{out}} - t_{\mathrm{recv}}$ is its own handling, which differs between the two clients because they deserialize on opposite sides of t_{recv} (Supplement S43.1). Every span referenced to t_{recv} therefore carries one client's payload parse and not the other's; Section VI-A says what that is worth.

b) One component is a proxy, not a chain: This distinction is load-bearing. The transport term of Equation 1 subtracts the producer's receipt of the broker's acknowledgment from the consumer's receipt of the record. The broker's append precedes both of those events, but neither of them precedes the other: they are two branches of one cause. The acknowledgment timestamp is therefore a *late, producer-side observation* of a broker-side event, and when the acknowledgment branch runs slower than the delivery branch the difference is negative without anything physically impossible having occurred. By contrast $t_{\mathrm{recv}} - t_{\mathrm{send}}$ and TTI itself are genuine chains: the producer sends, the broker appends, the consumer receives. Section V-A reports all four spans, and they behave completely differently.

B. The consistency check

We fix the rule before the final campaign and apply it to every run, including those whose results looked reasonable. **A run is rejected if more than 1% of its events carry a negative value in any latency component, or if the median of any component is negative.** A condition is usable only if all of its runs survive; where partly rejected we report the retention rate. The one-per-cent figure is a threshold, not a discovery. The supplement sweeps it from 0 to 20%, and no condition behind our first result is usable anywhere in that range, so the withdrawal does not turn on where the line was drawn.

The justification is not that a negative is impossible, because for the transport proxy it is not. It is that a negative proves the reference timestamp cannot serve as the origin of that event's flight. A run whose reference is unusable on more than one event in a hundred cannot report a latency, whatever the cause. The check is a *necessary* condition, not a validation procedure: a corpus that fails is unusable, and one that passes has cleared the cheapest available bar and nothing more. We use the sign as a gate rather than as an estimator, although the same constraint supports estimation, since offset and skew can

be recovered from a delay envelope [18]. That was deliberate: an estimator corrects a record, a gate declines to publish from a run whose instrument demonstrably failed, and after this check destroyed our own first result we preferred declining.

C. A model of the failure

A negative span is not a clock failure but an ordering failure between two timestamping delays, and the model that says so is two equations long. Every timestamp is taken some time after the physical event it claims to mark, because the thread that reads the clock must first be scheduled. This is not the cost of the clock call, which is small and separately measurable [16]; it is the delay before the call runs at all. Only the *difference* of two such delays matters: if both timestamps are late by the same amount, one has not measured a delay, one has invented a time zone. That difference needs no model; the timestamps of Equation 1 give it directly, per event and assuming no ordering:

$$S \;=\; D - A, \qquad D \equiv t_{\mathrm{recv}} - t_{\mathrm{send}}, \quad A \equiv t_{\mathrm{ack}} - t_{\mathrm{send}}. \tag{2}$$

So the span inverts exactly when the lag exceeds the delivery:

$$\Pr[S < 0] \;=\; \Pr[A > D]. \tag{3}$$

Figure 2 names the four timestamps; Supplement S45 maps them onto the span each metric covers.

Equation 3 is an identity, not a fit: both sides count $62{,}264$ over the $738{,}730$ events, and the build fails if they part. **Negative span probability is a property of the quantity being measured, not only of the machine**: a fixed distribution of A overlaps a small D almost entirely and a large one hardly at all. That is why our network-delay arm, transport in tens of seconds, passes every run, while same-hardware arms measuring one millisecond fail wholesale.

D. Statistics

Every proportion below is a count over a stated denominator with a Wilson score 95% interval, not Wald: at the real-time arms' rates the normal approximation is shifted low and covers below its nominal rate. At the smallest arm, 10 of $2{,}985$, Wald gives $[0.0013, 0.0054]$ against Wilson's $[0.0018, 0.0062]$. Holm correction is applied within a named family, and the corrected value is the one the tables display. Tail indices are estimated on the traced histogram by grouped maximum likelihood and by an exceedance ratio, never by a slope through survival points, because the buckets are interval-censored. The grouped estimator on log-spaced bins is Virkar and Clauset's, the Hill estimator for binned data, and we use their bootstrap for goodness of fit [32]. The remaining estimators are inventoried in Supplement S49. All interval arithmetic is recomputed from the committed artifacts at build time by `scripts/stat_intervals.py` and `scripts/tail_index_traced.py`, and the tables reporting it are generated rather than transcribed.

IV. Mode B: A Benchmark That Deletes Its Own Samples

The cross-broker benchmark we audit computes its reported latency distribution from a subset of the samples it collected. It does not record how large that subset was, and the size of the subset is determined by the send rate. Everything measured below is the OpenMessaging Benchmark in embedded mode, where driver and workers run in one JVM; its distributed mode is discussed in Section VI-D.

A. The guard, and its origin

In `LocalWorker.java`:294 the end-to-end latency is `now - publishTimestamp`, where `now` is read in the consumer and `publishTimestamp` is Kafka's producer-set `CreateTime`, itself a millisecond value. That difference is then promoted to microseconds (Supplement S36 quotes the call). So the guard in `WorkerStats.java`:95 — `if (endToEndLatencyMicros > 0)` — tests a variable named for microseconds that can only ever hold integer multiples of a thousand of them. The message is counted as received a few lines earlier, but a non-positive latency is dropped and no counter records how many. Over the campaign of Section IV-B that silence covers $11{,}532{,}492$ samples: Houdini at least took a bow.

The guard's origin is documented and its reasoning is sound. It was added in 2018 to stop negative values reaching the histogram, on the ground that end-to-end latency "is calculated based on time difference of `System.currentTimeMillis()` on two different machines" and "can be negative" [33]. The recorder is an `HdrHistogram Recorder`, which rejects negative values outright. Filtering for positives is the reasonable local fix, defensive rather than evasive. Its non-local consequences are two, neither considered at the time. First, `> 0` also deletes every sample that computes to exactly zero. Because `System.currentTimeMillis()` has millisecond resolution, any delivery faster than one millisecond does compute to zero; on co-located paths, where our own transport measures 0.1–0.5 ms, that is most of them. Second, the one observation which would prove the instrument had failed is the exact observation the guard removes. Section IV-D finds the same disposal, uncounted, in four independent tools.

The guard's scope is the most interesting thing about it. It is on the end-to-end path, which timestamps in milliseconds; the same harness measures its publish span with a nanosecond clock and records every sample unconditionally, needing no guard because that span is taken inside one process. The coarse clock and the filter both landed on the one span that crosses processes.

a) The consequence, measured: Of the 75 instrumented cells whose own summary we captured, 71 report a median of 1.0 or 2.0 ms, and across those the benchmark computes its reported distribution from between $\mathbf{0.36}\%$ and $\mathbf{100}\%$ of the samples it takes. Two replicates of one cell — 200 B payload at 500 msg/s, same host — kept 431 and $120{,}434$ samples, a 279-fold difference, and both reported a median of 1.0 ms with nothing in the output to distinguish them.

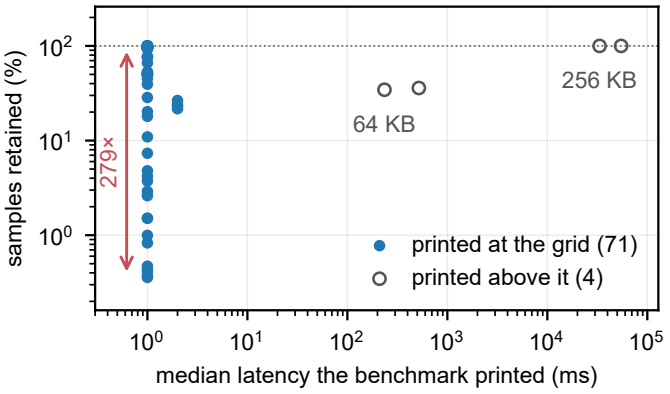

Fig. 3. **What the benchmark prints against what it kept in order to print it**, over every instrumented cell. At the quantum the printed median carries no information about retention: 71 cells report 1.0 or 2.0 ms while the retained fraction spans 279×. Away from the quantum the harness behaves, which is the point — the failure is confined to flights shorter than the instrument.

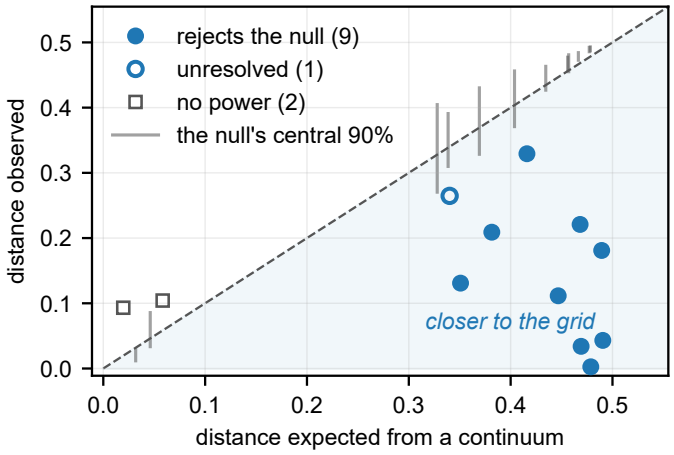

Fig. 4. **Grid membership, as a set.** Each arm's observed mean distance to the nearest grid vertex against the distance a continuum would give, which is the diagonal. The gray bar beside each arm, offset a little to clear the diagonal and mirrored for the two arms it would otherwise push onto the axis, is that arm's own null, simulated — *uncorrected*. The test is one-sided towards the grid, so a marker *below* its bar rejects that null and a marker above one does not; that is what lets the picture carry the test rather than illustrate it. Filled circles also survive Holm correction. The open circle sits *below* its bar too, 0.2647 against a band opening at 0.2680, but at $p = 0.044$ raw and 0.131 corrected it is the one powered arm that does not reject: correction is what the picture cannot draw. The 2 squares are the arms where the two hypotheses coincide and no test has power; their bars collapse towards zero and *the sign carries nothing* there. Per-arm corrected p-values are in Supplement S46.

Three uninstrumented runs print p50, p95, p99 and max all equal to 1.0 to three decimal places. The full ledger holds 223 instrumented runs, 11,532,492 discarded samples and 41,403 negatives, and over it the retained fraction falls as low as 0.0044% — four samples kept of ninety thousand. Figure 3 puts the two quantities on one pair of axes. The printed median is pinned at the quantum while retention moves across two and a half orders of magnitude. The only cells that escape are the four whose payload is large enough that the quantum stops binding.

That discarded fraction is the one time-interval averaging measures with [28]: reporting a distribution conditioned on it, without reporting it, keeps the residue and throws away the estimator.

B. Phase, and the grid it produces

Publish latency across these cells takes only two values, 0.3 and 0.4 ms, while retention varies 279-fold. A two-valued predictor cannot account for a range like that whatever its correlation, and the correlation it does have (+0.31, Fisher +0.08–+0.51, over 71 cells) carries the sign Equation 4 predicts. What moves retention is where the producer's send instants fall against the millisecond grid. A sample survives when its delivery crosses a tick boundary, which for a delivery shorter than one tick τ happens with probability T_{true}/τ (Supplement S51 draws the geometry):

$$\text{retention} = \min\left(1, \frac{T_{\text{true}}}{\tau}\right) \qquad \text{(uniform phases).} \quad (4)$$

All seven incommensurate arms sit near 50% — medians 47.2 to 51.5 — implying $T_{\text{true}} \approx 0.5$ ms at $\tau = 1$ ms.

a) Commensurability is not binary: Phases are not uniform when the send interval is commensurate with the tick. Write the send interval over the tick in lowest terms as $\Delta/\tau = p/q$ with $\gcd(p, q) = 1$. A producer then visits exactly q distinct phases. Retention is a count out of q and can take only the $q{+}1$ values $0, 1/q, \ldots, 1$, and a run lands on one of the two bracketing T_{true}/τ. Coarser grids are less reproducible, $q = 1$ being the all-or-nothing corner and an incommensurate rate effectively continuous (Supplement S23). The replicate spread follows:

$$\text{spread} \longrightarrow \begin{cases} 100/q & \text{when } T_{\text{true}}/\tau \text{ sits mid-cell,} \\ 0 & \text{when } T_{\text{true}}/\tau \text{ sits on a grid point.} \end{cases} \quad (5)$$

Both branches are limits. The lower is not zero because a nominally commensurate rate is not exactly commensurate; the two arms that sit on a vertex spread by roughly three tenths of the mid-cell width, too few to fix a constant (Supplement S23).

A round send rate is the worst possible choice, and exactness decides it: Supplement S23 runs from the $q = 1$ rate whose replicates span almost the whole range to the one sitting 0.00014 from a vertex and behaving as fully continuous.

C. The grid as inference

A table of spreads is not a test, so we state one: the mean normalized distance from an arm's replicates to the nearest q-grid vertex, against the within-arm permutation null a continuum implies. Supplements S54 and S46 give the construction and every arm with $n \geq 5$. 9 of the 10 powered arms reject the continuum null at $\alpha = 0.05$ after Holm correction within the family of 12. The tenth, 900 msg/s, rejects before correction ($p = 0.044$) and not after ($p = 0.131$); we report it as unresolved rather than as support. The remaining 2 arms, 400 and 800 msg/s, have no power by construction: where T_{true}/τ sits on a vertex the two predictions coincide, so the distance is replicate noise and which side of the diagonal it falls on is not evidence either way (Fig. 4).

a) A pre-registered confirmation: Payload moves T_{true}, hence the grid position, and Equation 5 dictates the arm's class with nothing else changed. The prediction was registered before the runs, and it came true: spread collapses to 13.6 at 32 KB, pinned near the $2/3$ vertex, and re-opens to 26.7 at 64 KB — the flat–full boundary crossed in both directions by one manipulated variable. One registered detail failed — the 32 KB pin sits at 69.2, not the predicted 66.67. Supplement S31 draws all three arms against their grid.

A larger campaign leaves per-send jitter rather than accumulation standing (Supplement S55).

D. Generality, and the sign channel

To show the law is not an artifact of one JVM, we reproduced the guard in an independent Python harness sharing no code with the benchmark. On one clock it recorded **zero** negative differences across $905{,}040$ samples, as a causal chain requires, and none across two hosts either (Supplement S10), while the grid behavior reappeared intact. At 457 msg/s — the incommensurate arm, the only one of the three whose own grid spread does not swamp the comparison — cross-host retention wandered from 13.4 to 26.9% over four replicates, where the same arm on one clock held to 0.98 points. Cross-host, the deleted fraction couples to the synchronization state and nothing records either.

Nor is the disposal confined to this harness, and deletion is not the only way the quantum wins. We audited ten tools at source across five languages and nine vendors, classifying each finding with the code that classified the benchmark. Five of the ten dispose of the sample without counting it, in three classes — a pattern rather than one project's oversight, and one of them is Apache Pulsar's performance client reaching the same design independently [34]. Apache Kafka's own bundled end-to-end tool keeps every sample and loses it anyway, filling the array its percentiles come from by integer division, so on a sub-millisecond path every one reports zero [8]. Two more keep their negatives and pass them on unfiltered, as RFC 7679 asks [35], [36]. The failure is not universal, and the claim that survives is the narrower one: *the tools that quantize or filter do so without counting*. Substitution is the worst of the three classes because it leaves nothing missing: retention computed downstream reads 100% and a value never measured enters the distribution — `fio` and `btt` substitute zero and one nanosecond, at the source lines Supplement S36 quotes. At the other end, one counts its discards in a metric whose description names the cause [37], which is what makes the rule of Section VI-B practical rather than aspirational. Supplement S36 gives the registry tool by tool, with the source line each finding sits on and one entry that corrects a misreading of our own.

The class is not confined to benchmarks: in November 2025 the audited broker's own coordinator metrics adopted the same disposal, clamping negative durations to zero with no counter and no log line [38]; its mechanism is a backward clock step, treated in Supplement S36.

Separating the discards by sign is what makes the guard's cost legible. Across the Kafka-driver corpus's 214 runs its $10{,}913{,}263$ discarded samples contain **not one negative**: the most negative value at any load, size or rate is $0\,\mu$s. Those discards are resolution failures, not clock failures — a distinction an unsigned counter cannot make. The same channel then caught the real thing in the Redis-driver replication's 9 runs: $41{,}403$ negatives, every one exactly $-1000\,\mu$s, the minimum the quantum can express, with $41{,}397$ of them in a single run where they were *half of all samples taken*, beside six kept.

That value is not a coincidence. The Redis driver's span is a difference between two millisecond-floored clocks on two hosts, and a sub-tick offset between two floored clocks can produce exactly one negative value, $-1000\,\mu$s, and no other. That is precisely what the corpus contains. The Kafka-driver span, floored the same way but taken inside one JVM, never inverts at all. Clock discipline was clean at every logged minute, which is the point — this failure needs no misbehaving clock, only two of them and a floor (Supplement S53).

V. Mode A: Why a Latency Comes Out Negative

The negatives in our own corpus are produced by scheduling, and we establish this by moving scheduling while holding everything else fixed.

A. What the audit rejected

The rule of Section III-B rejects 862 of $1{,}382$ runs on the workstation testbed (62.4%) and 459 of 884 on the cloud testbed (51.9%), every run behind our first result among them: on that first testbed 8 of 76 conditions survive. At saturation Kafka's median transport proxy reads -6.4 ms at one hundred connections.

Table I separates the components over a wider population than the audit: every cloud run that produced matched events, $5{,}913$ of them, rather than the 884 behind a reported result — a condemned run answers which span inverts as well as a clean one does, so none is excluded. The acknowledgment-referenced proxy is negative on $62{,}264$ of $738{,}730$ events (8.43%; Wilson intervals are under 0.1 points here and omitted hereafter), and $3{,}976$ runs exceed the one-per-cent threshold on it. The send-referenced span and TTI, which are causal chains, are negative on **no event at all**. $62{,}264$ messages arrived before they were sent. That is either a Nobel Prize or a scheduling artifact, and we regret to report the second. Nothing in this corpus arrived before it was sent; the reference timestamp was late. The rejections are therefore evidence of an unusable reference rather than of impossible physics, which is precisely what the gate is for.

The sign is a threshold, so the negatives count only the worst of the displacement. The acknowledgment lag exceeds half the delivery time on 74.5% of events and a tenth of it on 95.2%, against the 8.4% where it exceeds the whole and the check fires. The two are correlated within a run (median 0.84 over 70 conditions), one scheduler causing both, which suppresses crossings without touching the displacement. A path above the quantum is not clean but quiet.

TABLE I
NEGATIVES BY SPAN AND BROKER (5,913 RUNS, 738,730 EVENTS, ONE CLOCK). ONLY THE ACKNOWLEDGMENT-ORIGIN SPAN INVERTS, AND IT DOES SO UNDER BOTH BROKERS: KAFKA ON 8.67% OF EVENTS, REDIS ON 8.19%. THE CAUSAL CHAINS ARE CLEAN UNDER EACH BROKER, SO THEIR PER-BROKER COLUMNS ARE DASHED RATHER THAN REPEAT A ZERO THREE TIMES.

Span	Origin timestamp	Kafka	Redis	All
$t_{\mathrm{recv}} - t_{\mathrm{ack}}$	acknowledgment	31,899	30,365	62,264
$t_{\mathrm{recv}} - t_{\mathrm{send}}$	send (chain)	—	—	0
$t_{\mathrm{out}} - t_{\mathrm{send}}$	send (chain)	—	—	0
TTI	emission	—	—	0

a) Why it looked correct: The rejected result was monotone in concurrency, significant after correction ($p = 9.0 \times 10^{-11}$), and passed every conventional check we knew: replicate counts, effect sizes, corrected significance, sanity plots. The acknowledgment timestamp is read only once its waiting thread is rescheduled, so under saturation the consumer has already recorded receipt; the send timestamp cannot be late that way, whichever client runs (Supplement S43).

b) What the check cannot catch: A great deal: load-induced inflation that stays positive is physically possible and needs a better instrument. Our own second withdrawal is the example — a twentyfold end-to-end gap that was a per-run start-up cost read as a per-event constant, causally consistent and wrong. Causal consistency is necessary, not sufficient.

B. A rare state, not a wide distribution

Each timestamping thread is either running, in which case its timestamp is late by a narrow jitter of width σ_c, or preempted and off-CPU. Write p for the probability of the second state and $G(\cdot)$ for the residual's survival function, and take the consumer's timestamp to lie in the core:

$$\Pr[\text{span} < 0 \mid T_{\mathrm{true}}] \;=\; p\, G(T_{\mathrm{true}}). \tag{6}$$

Load moves p and does not fix it — two geometries at one ρ, rates twofold apart (Table II, lower panel) — so we write p and not $p(\rho)$, and the model's content is on the flight axis rather than the load one (Supplement S2). That last step is the leading-order case, and not because the two states are independent. A stall delaying both branches cancels in $S = D - A$, so what it sets aside is invisible rather than rare. Load changes how *often* the thread that must record the time is not running, not how wide the ordinary jitter is. Going from idle to the knee multiplies the fitted core width σ_c by 5 and the negative-span rate — the mass of A beyond D — by 61. The rate has a ceiling below one: fully saturated it reaches 0.37, so even then most timestamps are taken by a thread that was running, or late by less than the flight.

C. The experiments that settle it

No pair of arms in Table II overlaps, and Supplement S47 draws them. Real-time priority on the timestamping threads moves occupancy while utilization stays fixed (Table II, top). The rate falls by factors of 39 [21–74] and 54 [33–86]. The brackets are Katz intervals on the ratio itself, wide because the collapsed arms are the sparse ones, and the Wilson intervals of the two arms are disjoint at both load levels ($z = 19.8$ and $z = 31.9$). Both residual rates land on the scale the model names in advance, $C_0 \approx 0.004$; over all 8 pairs the manipulated arm runs from 0.0017 to 0.0144, so C_0 is where the rate settles and not a bound it respects. *This refutes every account in which the negative-span rate is a function of utilization*, including the queueing form we ourselves adopted: ρ did not change and the rate changed fiftyfold. Across 8 matched pairs spanning 60 to 95% load the factor ranges 7–80×, every pair with disjoint intervals. Supplement S47 reports each one, with the campaign it came from.

TABLE II
THE MECHANISM, BY MANIPULATION (2,985 MATCHED EVENTS PER CELL). TOP: TIMESTAMPING THREADS MOVED TO `SCHED_FIFO` WITH BACKGROUND LOAD UNCHANGED; ACHIEVED UTILIZATION MATCHES TO WITHIN 0.0006 BETWEEN ARMS, SO OCCUPANCY ALONE MOVED. BOTTOM: IDENTICAL UTILIZATION REACHED BY TWO CORE GEOMETRIES ($\rho = 0.7531$ IN ALL FOUR k=6 ARMS); A FUNCTION OF ρ RETURNS ONE VALUE FOR ONE INPUT. SUPPLEMENT S47 PRINTS BOTH ARMS' ACHIEVED ρ, PAIR BY PAIR.

Manipulation	Arm	Rate	95% CI	Factor (z)
Priority, 75%	ordinary	0.1320	0.1203–0.1446	39× (19.8)
	real-time	0.0034	0.0018–0.0062	
Priority, 88%	ordinary	0.3049	0.2886–0.3216	54× (31.9)
	real-time	0.0057	0.0036–0.0091	
Geometry, original	concentrated	0.0824	0.0731–0.0928	2.07× (10.3)
	spread	0.1709	0.1578–0.1848	
Geometry, repl.	concentrated	0.0600	0.0520–0.0691	2.05× (8.4)
	spread	0.1229	0.1117–0.1352	

Three further manipulations agree. At $k = 6$ both load geometries sit at $\rho = 0.7531$, identical to four decimals, and the negative-span rates differ by 2.07× [1.80–2.39] ($z = 10.3$), reproduced at 2.05× [1.73–2.43] by a full replication. Multi-server queueing expects geometry to matter at fixed ρ, so this will not surprise a queueing-literate reader. What it rules out is the single-parameter form we had adopted and pre-registered, where ρ alone fixes the rate. Payload padding lengthened transport 76.9× while the negative-span rate *fell* 4.1× [3.5–4.8], monotonically, at a 0.012 utilization spread — the direction Equation 3 predicts and the opposite of a stress account. Finally, a kernel trace on the `sched_wakeup` and `sched_switch` tracepoints [39] predicts that rate to within a third across 3 arms at two load levels — ratios $0.78, 1.06, 1.32$, no consistent sign, nothing fitted. It is filtered to the harness processes and shares no instrument, estimator or data with the application-level rate. The probe is not free: 0.272 untraced against 0.231 traced ($z = 3.6$), and the comparison uses the traced arm's own rate, so 0.272 is the machine's (Supplement S43.4).

a) The load-axis predictions, and their failure: The knee sweep placed conditions where the candidate load laws diverge and both failed, because both were parameterized in ρ and ρ is not the variable (Supplement S2).

D. The stall spectrum, and the scheduler's slice

That this distribution is not a single heavy tail is known — B. Gregg reported it as bimodal in the post introducing

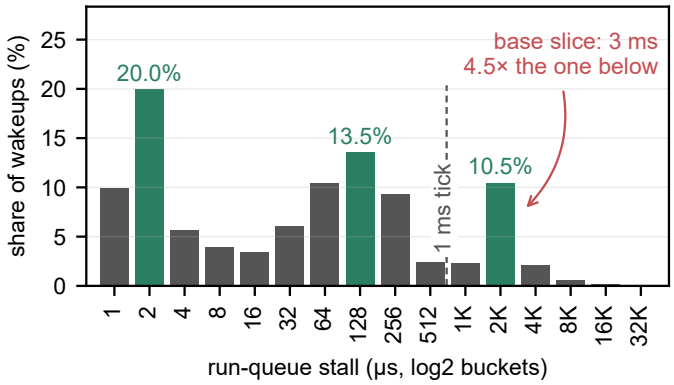

Fig. 5. **The traced run-queue stall distribution is trimodal.** 551,956 wakeups, `bpftrace` $\log_2$ buckets, modes shaded. The jitter core sits at a few microseconds and a second mode at 128–256 μs; the third, holding 10.5% of all wakeups, lands on the scheduler's derived base slice of 3 ms. A power law of index above one — the fitted index is 2.04 — would have to descend monotonically across this axis.

`runqlat`, the tool we use [39]. The rigorous treatment of that measurement is `timerlat`, which decomposes scheduling latency into its interrupt and thread components and separates the tracer's own cost from them [40]. What we add is which modes, and why the upper one sits where it does.

Raising the payload from 0 to 256 KB lengthens the true transport 77×, and over that range the negative-span rate falls monotonically with the delivery D. Log–log least squares over the four payload levels returns a slope of -0.34 (-0.44 to -0.23, Student-t at two degrees of freedom). The fit and the figure drawing it are in Supplement S32; four points do not earn an equation here. An earlier version read the kernel trace as an independent confirmation of it. We withdraw the claim, because estimating the traced index properly is what refuted it.

Two estimators of the traced tail index disagree sixfold, and a bootstrap goodness-of-fit rejects the fitted power law ($p < 0.0004$ over 2,500 replicates drawn from it); both are in Supplement S32. The reason needs no fitting. Over the 551,956 traced wakeups the counts do not descend monotonically, and on $\log_2$ buckets a power law of index above one must. The fitted index is 2.04. What the counts have instead is 3 local maxima. The last is a *mode* at 2–4 ms, holding 10.5% of all wakeups and standing 4.5× its lower neighbor. Above it the survival is light: grouped maximum likelihood beyond 4 ms gives 2.04 (2.00–2.07), a finite variance. Neither that share nor that position has, to our knowledge, been attributed to a scheduler constant before. Figure 5 is the histogram, and the shape is the argument: no monotone curve passes through it.

That mode is the preempted state of Equation 6, at a scale the scheduler's own maintainers had already named: with run-to-parity a task's lag is held within "$[-(\text{slice} + \text{tick}) : (\text{slice}+\text{tick})]$" [41]. The histogram adds that this is not merely a worst case but where a tenth of all wakeups land. The slice is *derived* rather than measured — the kernel's published rule [42] gives $4 \times 0.75\,\text{ms} = 3$ ms on the 8 cores the campaign's own load geometry recovers, over a 1 ms *scheduler* tick, and no scheduler tunable is written anywhere in the archived campaign (Supplement S44). That tick is the kernel's timer period, not the timestamp quantum of Section IV. Both are 1 ms here, which is a coincidence of configuration, so we name the scheduler's explicitly wherever the two are in play. A thread descheduled at the wrong instant therefore reads its clock about a slice late, which on a path whose true transport is 0.1–0.5 ms is six to thirty times the flight being measured. The scheduler tick is why the mode is a band rather than a spike. A mean over this distribution is dominated by a mode the operator never sees, so a mean-based counter cannot flag this failure.

VI. Discussion

Both failures are cheap to detect and neither is detected, so the remedy is a reporting rule rather than a better clock.

A. What the brokers actually do

Once the failed runs are removed, the broker comparison that motivated the work is small and one configuration choice dwarfs it. On gated artifacts the two sit within a millisecond on the transport proxy (TOST against a 1 ms margin, $p < 0.001$, across 3 levels; the brokers lose runs to the audit at different rates, and Supplement S23 bounds that at the same margin), so it is not a purchasing argument. The one condition that reverses the ordering is a consumption pattern rather than the broker, and a co-located testbed — the standard evaluation environment — hides the choice that decides the outcome (Supplement S48).

Our own instrument is not symmetric across that comparison, and the rule below says to declare it. The transport proxy ends at t_{recv}, and Kafka's client deserializes the payload before that timestamp while Redis's deserializes after it, so the span carries one client's parse and not the other's: 19.5 μs, or 2.4% of Redis's own median delivery, which is the denominator it sits inside. That is a fiftieth of the equivalence margin above and cannot move it, and it runs *against* the per-broker difference of Table I rather than producing it. TTI is free of it, because both parses fall inside TTI (Supplement S43.1).

B. For benchmark authors

SPEC and TPC judge a benchmark on five criteria [3]. The instrument we audit passes the half of *reproducibility* a reader checks — run-to-run consistency at a fixed configuration — *because* it fails *verifiability*. A median from 0.36% of the samples is the more stable for having had the variation deleted, and a second tester who picks a different send rate measures a different sample. Every rule below instances the criterion that can see this.

Report a physical-consistency audit. Any cross-process timestamp difference admits a sign check that costs nothing, and every run it rejected here had passed every conventional check we knew. Better instruments exist [13], [14], [15]. Whatever the instrument, check its output against causality and publish the rejection rate, as network clocks have for thirty years [18], [1]. The discipline is tested only by a campaign the rate costs something, and ours cost us a result.

Name the span you are subtracting. A difference of two timestamps is a latency only if the first event causally precedes the second, and an acknowledgment at the producer does not precede a record at the consumer (Section III-A). We caught our own instance only on counting the send-referenced span separately (Table I).

Where the span cannot be restamped, recover the displacement rather than discard it. Equation 2 holds per event and A is producer-side on one clock, so a harness that records its own acknowledgment lag recovers the delivery it failed to measure. The recovered median lands within 3.4% of the true one on runs the check passes and 12.2% on the runs it *rejects* — the population a remedy tried only against clean runs never meets.

Two delays with one cause are not independent. Timestamps waiting on one scheduler move together ($\rho = 0.84$), so multiplying their probabilities misprices the answer: assuming independence here overpredicts the negative-span rate in all 70 conditions, by a median factor of 12.4. What rescues the subtraction is cancellation, not rarity — a stall delaying both branches leaves $S = D - A$ untouched.

Know where your own path sits on the exposure curve, and how wide it is. The displacement is a scheduling quantity and does not grow because a message travelled further, so the relative error is A/D and falls as $1/D$: 7% at a 10 ms path, 1% at 100 ms. Read it the other way and it is a warning: 72% at 1 ms, and below 0.72 ms the displacement exceeds the path itself — while sub-millisecond medians are routine in broker benchmarking (Supplement S48). Those are one lag: $A = 725\,\mu$s, our median. Quoting a curve by its middle is this paper's own mistake one level up, and A is no constant — ours spans 500–1900 μs across conditions, putting the 10 ms error at 7–19% and the crossover at 0.72–1.90 ms. Measure your own; Supplement S48 bands every row. The curve is a statement about acknowledgment-referenced spans; a harness that subtracts a send timestamp reports the delivery and is not on it, which is the whole return on naming the span. Two *identical* systems whose clients timestamp in different places are separated by the difference of their lags, a reported gap of 11% at 10 ms, and the sign check fires at neither. On this corpus the reported span is 24% of the delivery it stands for, understating it $4.2\times$ — a median of per-condition ratios, which is not what differencing the two published medians gives (Supplement S48). The two brokers' clients timestamp differently. **The instrument inflates the answer a comparison exists to give by about seventy per cent** — $1.7\times$ over 24 matched workloads, interquartile range 1.2–2.0, and it *narrows* the gap on 21% of them — a property of the clients, not of the brokers.

Count your events before you quote a percentile. A median over as few as seven events per run is not a property of the system, and repetition will not tell you so.

Make the timestamping path unpreemptable, and say that you did. The one mitigation our measurements support directly: real-time priority on the timestamping threads cut the negative-span rate 7–80$\times$ at unchanged background load, and busy-polling a dedicated core should buy the same for a burned core. Two caveats: the floor is not zero, so the check stays; and a real-time or spinning timestamping thread changes the system it measures, so where that is unacceptable, report retention instead.

Dither the send instant, and publish the retained fraction. Randomizing probe timing to avoid synchronizing with a periodic phenomenon is long-standing advice [23], [24]; what we add is why — the phenomenon synchronized with is the timestamp quantum itself. Publish beside the latency the retention rate, the timestamp resolution and the synchronization state, none recoverable afterwards: a summary from 0.36% of samples is typographically identical to one from all. Neither half of the rule is ours. Both are the detail beyond the metric that verifiability asks for, and three measurement standards already require some part of it: OWAMP has attached an error estimate to every timestamp since 2006, and records a packet timestamped in the future unaltered rather than discarding it [43]. The sign is the channel: negative means the reference timestamp failed, computes-to-zero means the resolution failed, and one unsigned total cannot distinguish them. Supplements S39 and S40 give the precedents in full.

Record the achieved rate, not the flag meant to produce it. Verify it per run against elapsed wall time: the send schedule is an experimental variable in Mode B, not a nuisance parameter, and we found runs where configured and achieved rates disagreed.

None of this is hypothetical: a 2026 framework for the same comparisons already timestamps in nanoseconds and subtracts a span that cannot invert, both choices made and neither argued for (Supplement S52.2).

C. The limits of a better clock

Hardware-timestamped PTP [12] would take two LAN hosts from our $67\,\mu$s of `chrony` agreement to sub-microsecond. Behind a one-millisecond timestamp, $67\,\mu$s and 70 ns produce byte-identical records: the guard deletes the same samples and the grid law is unchanged. The remedies are therefore ordered: record at native resolution, count what is discarded, dither the send instant, and only then is synchronization quality the frontier — our cross-host retention wandered 13.4–26.9% under clocks agreeing to well under a tick (Section IV-D). Resolution is the lesser half, and a distinct one: a microsecond timestamp retires Equation 4's deletions here and moves the grid to faster paths, but the guard survives it and genuine negatives do too, and the obvious redesign does not help either (Supplements S37 and S43.6). A better clock alone therefore converts an uncounted *resolution* failure into an uncounted *instrument* one.

D. Threats and limitations

A negative span proves something is wrong; attributing it to scheduling is an inference, separated from its rivals by manipulation, with the eliminations listed at the end of Section V. Beyond that list, the clustering of consecutive late wakes ($z = -6.9$ at the co-located floor, at every load including none) is the signature of a shared scheduling delay, not independent kernel granularity. The `chrony` daemon reports the inter-host offset of Section III-A as comfortably

sub-tick, but bounds its own error at 4–7 ms per host. The two worst of those bounds sum to 12 ms against a 1 ms timestamp. The cross-host channel therefore stays open rather than ruled out. It did not produce the Redis-driver negatives, which the driver's own timestamp accounts for exactly (Section IV-D), but it is why we claim no cross-host bound in general.

a) Scope, and the alternatives eliminated: Three widely used runtimes document this structure without drawing its consequence for a cross-process subtraction (Supplement S38). The mechanism requires a timestamping path that can be preempted. Designs that pin a thread to a dedicated core and busy-poll, or that timestamp in hardware, remove the state Equation 6 depends on, and we make no claim about them. Within our own setting the alternatives are each eliminated. Clock skew: one clock by construction on the rejecting arms, and 0 negatives in 905,040 two-clock samples. Cross-CPU incoherence under migration: below. Utilization: identical ρ, rates twofold apart. Timer noise: `SCHED_FIFO` at fixed utilization collapses the rate across 8 pairs, and noise does not respect scheduling class. Stress: transport 77× longer *lowers* the rate. Direct observation: a `sched_switch` histogram predicts the rate to within a third, unfitted. One rival is not the scheduler's — the timestamp is a CPython call, so it waits on the interpreter lock too, invisibly to a tracepoint probe. That one is bounded because the kernel-only estimator brackets the observed rate $(0.78, 1.06, 1.32)$ instead of under-predicting (Supplement S43.4).

Clock incoherence across vCPUs needs its own answer, our machines being virtual [44], and gets one twice over. The probe's 90 ns increment admits only `kvm-clock` or `tsc`, both of which the kernel uses solely where cross-CPU coherence is asserted. And the corpus closes the channel again on headroom, not on shared endpoints: the send-referenced span holds a floor 455× too high to explain the deepest negative span (Supplement S42).

The mechanism's constants are those of one EEVDF-era kernel (6.8); CFS-era schedulers, the regime Lozi et al. document, may shift them. The deletion law is demonstrated on one benchmark and reproduced in one independent harness, and the five tools of Section IV-D are readings of source rather than measured deployments. Nor is the guard confined to the tool we measured: 40 of the 40 public forks we could read carry the expression unchanged, surveyed 2026-08-24. Its distributed mode failed on every attempt, so that tool's cross-host exposure rests on source reading and our own two-host harness (Supplement S4). Exposure depends on a deployment's path latency and producer rate, which is why we recommend publishing a retention rate rather than calling any published number wrong.

VII. Conclusion

A latency benchmark measures the system only while the instrument is faster than the thing being measured. Below that line the two failures reported here appear, invisible in the output. Two timestamps written by threads that wait for a core independently invert the flight between them whenever those waits differ by more than it. A timestamp quantized to a millisecond, filtered for positivity, deletes most of a sub-millisecond path by arithmetic the operator never sees. Neither needs a better clock, and neither is expensive to detect. Check the spans whose endpoints are timestamped in different places, count what your instrument throws away, and publish the retention rate beside the number.

Acknowledgment

The authors thank J. Kunkel, for the same-clock protocol contrast, the offset-estimation route and the measurement-model figure; M. Luthra, for the kernel-tracing pointers; M. Kogias, for the probe-placement argument; and L. Tratt, for the question about timestamp resolution answered in Section IV.

In accordance with IEEE policy, Anthropic's Claude assisted with code development and verification, manuscript editing, and literature search. Experimental design, scientific claims, and interpretation are the authors', and all reported quantities are reproducibly computed from the committed data and archived code.

Artifact Availability

Code, analysis and manuscript are archived at 10.5281/zenodo.21650031, the measurement dataset at 10.5281/zenodo.21650064; both resolve to the current version, `v3.0.0`. Every number here is recomputed from the committed data by the archived code at build time, and the test suite fails if the text and the data disagree.

References

[1] V. Paxson, "Strategies for sound Internet measurement," in *Proc. ACM Internet Measurement Conf. (IMC)*. ACM, 2004, pp. 263–271, physical impossibility as grounds for discarding a trace.

[2] Standard Performance Evaluation Corporation, "SPECjbb2015 run and reporting rules, v1.02," 2018, §3.4.5; an initial clock offset beyond ten minutes voids the run.

[3] J. von Kistowski, J. A. Arnold, K. Huppler, K.-D. Lange, J. L. Henning, and P. Cao, "How to build a benchmark," in *Proc. 6th ACM/SPEC Int. Conf. on Performance Engineering (ICPE)*, 2015, pp. 333–336, section 3, the five quality criteria of a standardized benchmark.

[4] OpenMessaging Project, Linux Foundation, "The OpenMessaging benchmark framework," 2018, available at openmessaging.cloud/docs/benchmarks/; accessed 19 August 2026.

[5] T. Mytkowicz, A. Diwan, M. Hauswirth, and P. F. Sweeney, "Producing wrong data without doing anything obviously wrong!" in *Proc. 14th Int. Conf. Architectural Support for Programming Languages and Operating Systems (ASPLOS)*. ACM, 2009, pp. 265–276.

[6] A. Georges, D. Buytaert, and L. Eeckhout, "Statistically rigorous Java performance evaluation," in *Proc. 22nd ACM SIGPLAN Conf. Object-Oriented Programming, Systems, Languages and Applications (OOPSLA)*. ACM, 2007, pp. 57–76.

[7] T. Hoefler and R. Belli, "Scientific benchmarking of parallel computing systems: Twelve ways to tell the masses when reporting performance results," in *Proc. Int. Conf. High Performance Computing, Networking, Storage and Analysis (SC)*. ACM, 2015, pp. 73:1–73:12.

[8] Apache Software Foundation, "Apache Kafka tools — `EndToEndLatency.java` and `ProducerPerformance.java`," Source repository, 2026, https://github.com/apache/kafka; the percentile array is filled by integer division, `latencies[i] = elapsed / 1000 / 1000`, while each sample prints at full precision. Read 2026-08-31.

[9] J. Karimov, T. Rabl, A. Katsifodimos, R. Samarev, H. Heiskanen, and V. Markl, "Benchmarking distributed stream data processing systems," in *Proc. IEEE 34th Int. Conf. Data Engineering (ICDE)*, 2018, pp. 1507–1518.

[10] G. Van Dongen and D. Van den Poel, "Evaluation of stream processing frameworks," *IEEE Trans. Parallel Distrib. Syst.*, vol. 31, no. 8, pp. 1845–1858, 2020.
[11] L. Lamport, "Time, clocks, and the ordering of events in a distributed system," *Commun. ACM*, vol. 21, no. 7, pp. 558–565, 1978.
[12] IEEE, "IEEE standard for a precision clock synchronization protocol for networked measurement and control systems," IEEE Std 1588-2019, 2020, revision of IEEE Std 1588-2008.
[13] S. Yang, J. Jeong, B. Scholz, and B. Burgstaller, "Cloudprofiler: TSC-based inter-node profiling and high-throughput data ingestion for cloud streaming workloads," *arXiv preprint arXiv:2205.09325*, 2022.
[14] M. Kogias, S. Mallon, and E. Bugnion, "Lancet: A self-correcting latency measuring tool," in *Proc. USENIX Annual Technical Conf. (ATC)*, 2019, pp. 881–896, §4.3: terminates and reports the results with an inconclusive verdict.
[15] R. Kundel, "P4STA: High precision network function benchmarking," in *Accelerating Network Functions Using Reconfigurable Hardware*, ser. Springer Theses. Springer Nature Switzerland, 2024, pp. 93–115.
[16] M. Kuperberg, M. Krogmann, and R. H. Reussner, "Metric-based selection of timer methods for accurate measurements," in *Proc. 2nd ACM/SPEC Int. Conf. Performance Engineering (ICPE)*, Mar. 2011, pp. 151–156.
[17] B. Villain, M. Davis, J. Ridoux, D. Veitch, and N. Normand, "Probing the latencies of software timestamping," in *Proc. IEEE Int. Symp. Precision Clock Synchronization for Measurement, Control and Communication (ISPCS)*, 2012, pp. 1–6, dTrace breakdown against a DAG hardware reference.
[18] V. Paxson, "On calibrating measurements of packet transit times," in *Proc. ACM SIGMETRICS*. ACM, 1998, pp. 11–21.
[19] A. Swami and N. Chougule, "Architecture dependent temporal observability under deployment interference in edge inference systems," *arXiv preprint arXiv:2605.17701*, 2026.
[20] A. Sharma, D. Shah, D. Lariviere, and H. ElBakoury, "Time, causality, and observability failures in distributed AI inference systems," *arXiv preprint arXiv:2604.21361*, Apr. 2026, open Compute Project, Unified Intelligent Infrastructure workstream.
[21] OpenMessaging Benchmark contributors, "Can the average publish latency be larger than end-to-end latency? (issue #216)," https://github.com/openmessaging/benchmark/issues/216, 2021, the anomaly attributed to inter-node skew, on a same-node report.
[22] J. C. Gust, R. M. Graham, and M. A. Lombardi, "Stopwatch and timer calibrations (2009 edition)," National Institute of Standards and Technology, NIST Special Publication 960-12, 2009, uncertainty budget carrying both operator reaction time and display resolution.
[23] V. Paxson, G. Almes, J. Mahdavi, and M. Mathis, "Framework for IP performance metrics," Internet Engineering Task Force, RFC 2330, May 1998.
[24] V. Raisanen, G. Grotefeld, and A. Morton, "Network performance measurement with periodic streams," Internet Engineering Task Force, RFC 3432, Nov. 2002.
[25] G. Tene, "How NOT to measure latency," 2015, talk, Strange Loop / QCon; coined "coordinated omission".
[26] M. Wiederhold, "Sub-millisecond latencies are reported as taking 0 milliseconds (issue #41)," Yahoo! Cloud Serving Benchmark, Oct. 2011, https://github.com/brianfrankcooper/YCSB/issues/41; fixed by pull request #42 and released in 0.1.4.
[27] C. Millsap and J. Holt, *Optimizing Oracle Performance*. Sebastopol, CA: O'Reilly, 2003, section 7.7, "Quantization Error".
[28] Hewlett-Packard, "Time interval averaging," Hewlett-Packard Company, Application Note 162-1, 1970, part no. 02-5952-0766; undated; the year is its *HP Journal* citation.
[29] J.-P. Lozi, B. Lepers, J. Funston, F. Gaud, V. Quéma, and A. Fedorova, "The Linux scheduler: a decade of wasted cores," in *Proc. 11th European Conf. Computer Systems (EuroSys)*. ACM, 2016, pp. 1–16.
[30] A. Chandrasekar and J. Kramberger, "Identifying and mitigating systemic measurement bias in production LLM inference benchmarks," *arXiv preprint arXiv:2605.24217v2*, 2026.
[31] S. Bradner, "Benchmarking terminology for network interconnection devices," Internet Engineering Task Force, RFC 1242, 1991, section 3.9 defines latency for cut-through devices, where the value can be negative because forwarding begins before reception completes.
[32] Y. Virkar and A. Clauset, "Power-law distributions in binned empirical data," *The Annals of Applied Statistics*, vol. 8, no. 1, pp. 89–119, 2014.
[33] S. Guo, "Avoid adding negative metric values (pull request #56)," OpenMessaging Benchmark, commit ebb5d6b8, Mar. 2018, https://github.com/openmessaging/benchmark/pull/56; accessed 18 August 2026.
[34] Apache Pulsar contributors, "`PerformanceConsumer.java`," Source repository, 2026, https://github.com/apache/pulsar. Read 2026-08-21.
[35] Broadcom, "RabbitMQ PerfTest — `Consumer.java`," Source repository, 2026, https://github.com/rabbitmq/rabbitmq-perf-test; the cross-process difference is passed to the metrics unfiltered. Read 2026-08-31.
[36] Synadia, "NATS cli — `cli/latency_command.go`," Source repository, 2026, https://github.com/nats-io/natscli; the wall-clock difference is appended unconditionally. Read 2026-08-31.
[37] IOP Systems, "Rezolus: systems performance telemetry — `scheduler_discarded_samples`," Source repository, 2026, https://github.com/iopsystems/rezolus; counts samples discarded for out-of-order timestamps. Read 2026-08-20.
[38] Apache Kafka contributors, "KAFKA-19888: Clamp negative values in coordinator histograms," Issue tracker and pull request #20986, 2025, https://issues.apache.org/jira/browse/KAFKA-19888; merged 2025-11-26, shipped in 4.2.0, backported to 4.0.2/4.1.2. Read 2026-08-21.
[39] B. Gregg, "Linux bcc/BPF run queue (scheduler) latency," https://www.brendangregg.com/blog/2016-10-08/linux-bcc-runqlat.html, Oct. 2016, introduces `runqlat`; reports the distribution as bimodal without attributing a mechanism.
[40] D. Bristot de Oliveira, D. Casini, J. Lelli, and T. Cucinotta, "Timerlat: Real-time Linux scheduling latency measurements, tracing, and analysis," *IEEE Trans. Comput.*, vol. 74, no. 8, pp. 2608–2620, 2025.
[41] V. Guittot, "`sched/fair`: Manage lag and run to parity with different slices," Linux kernel mailing list, Jul. 2025, states the base-slice protection and the slice-plus-tick lag bound.
[42] Z. Zhou, "sched: Reduce the default slice to avoid tasks getting an extra tick," Linux kernel mailing list, commit 2ae891b82695, 2025, reviewed-by V. Guittot; reduces the constant to `0.70` in v6.15.
[43] S. Shalunov, B. Teitelbaum, A. Karp, J. W. Boote, and M. J. Zekauskas, "A one-way active measurement protocol (OWAMP)," Internet Engineering Task Force, RFC 4656, Sep. 2006.
[44] Linux Kernel Documentation, "KVM-specific MSRs: `MSR_KVM_SYSTEM_TIME_NEW`," https://docs.kernel.org/virt/kvm/x86/msr.html, 2026, flag bit 0, CPUID `0x40000001` bit 24. Read 2026-08-21.

Gustavo Pedro Ricou received the B.Sc. degree in physics from the Universidade do Porto, Portugal, and specialized in nuclear fusion science at the Universität Stuttgart, Germany, and the Université de Lorraine, France. He holds an M.Sc. in software design with cloud-native computing from the Technological University of the Shannon, with first-class honors, and is currently pursuing an M.Sc. in statistics and data science at Trinity College Dublin, Ireland. He is a Software Engineer with BoscaSports. His research concerns measurement validity — what a reported number is really evidence of — in computer systems, environmental measurement and sports analytics, using pre-registration and physical-consistency auditing.

David Gregg received the Ph.D. degree in computer science from the Technische Universität Wien, Vienna, Austria, in 2001, for work on compiling for instruction-level-parallel architectures. He is a Professor in the School of Computer Science and Statistics, Trinity College Dublin, Ireland, and a Fellow of the Irish Computer Society. His research concerns compilers, algorithms and software optimization for multicore and vector computers, and low-resource techniques for deep neural networks.

Romaric Duvignau received the Ph.D. degree in computer science from the University of Bordeaux, France, in 2015, for work on the maintenance and simulation of dynamic random graphs. He is an Associate Professor and Head of the Network and Systems unit at Chalmers University of Technology, Gothenburg, Sweden. His research concerns distributed and data-driven algorithms, the monitoring of large-scale distributed systems, and stream analytics for cyber-physical systems.

Nikolas Herbst received the Ph.D. degree from the University of Würzburg, Germany, in 2018. He is a research group leader at the Chair of Software Engineering, University of Würzburg, and serves as elected vice-chair of the SPEC Research Cloud Group. His research topics include predictive data analysis, elasticity, auto-scaling, resource management, and performance evaluation of virtualized environments.